arXiv:1710.02410v1 [cs.RO] 6 Oct 2017

End-to-end Driving via Conditional Imitation Learning

Felipe Codevilla[1,2] Matthias Müller[1,3] Alexey Dosovitskiy[1] Antonio López[2] Vladlen Koltun[1]



(a) Aerial view of test environment (b) Vision-based driving, view from onboard camera (c) Side view of vehicle

Fig. 1. Conditional imitation learning allows an autonomous vehicle trained end-to-end to be directed by high-level commands. (a) We train and evaluate robotic vehicles in the physical world (top) and in simulated urban environments (bottom). (b) The vehicles drive based on video from a forward-facing onboard camera. At the time these images were taken, the vehicle was given the command "turn right at the next intersection". (c) The trained controller handles sensorimotor coordination (staying on the road, avoiding collisions) and follows the provided commands.

***Abstract*— Deep networks trained on demonstrations of human driving have learned to follow roads and avoid obstacles. However, driving policies trained via imitation learning cannot be controlled at test time. A vehicle trained end-to-end to imitate an expert cannot be guided to take a specific turn at an upcoming intersection. This limits the utility of such systems. We propose to condition imitation learning on high-level command input. At test time, the learned driving policy functions as a chauffeur that handles sensorimotor coordination but continues to respond to navigational commands. We evaluate different architectures for conditional imitation learning in vision-based driving. We conduct experiments in realistic three-dimensional simulations of urban driving and on a 1/5 scale robotic truck that is trained to drive in a residential area. Both systems drive based on visual input yet remain responsive to high-level navigational commands. Experimental results demonstrate that the presented approach significantly outperforms a number of baselines.**

I. Introduction

Imitation learning is receiving renewed interest as a promising approach to training autonomous driving systems. Demonstrations of human driving are easy to collect at scale. Given such demonstrations, imitation learning can be used to train a model that maps perceptual inputs to control commands; for example, mapping camera images to steering and acceleration. This approach has been applied to lane following [27], [4] and off-road obstacle avoidance [22]. However, these systems have characteristic limitations. For example, the network trained by Bojarski et al. was given control over lane and road following only. When a lane change or a turn from one road to another were required, the human driver had to take control [4].

Why has imitation learning not scaled up to fully autonomous urban driving? One limitation is in the assumption that the optimal action can be inferred from the perceptual input alone. When a car approaches an intersection, the camera input is not sufficient to predict whether the car should turn left, right, or go straight. Mathematically, the mapping from the image to the control command is no longer a function. Fitting a function approximator is thus bound to run into difficulties. This had already been observed in the work of Pomerleau: "Currently upon reaching a fork, the network may output two widely discrepant travel directions, one for each choice. The result is often an oscillation in the dictated travel direction" [27]. Even if the network can resolve the ambiguity in favor of some course of action, it may not be the one desired by the passenger, who lacks a communication channel for controlling the network itself.

In this paper, we address this challenge with conditional imitation learning. At training time, the model is given not only the perceptual input and the control signal, but

[1]Intel Labs
[2]Universitat Autonoma de Barcelona
[3]King Abdullah University of Science & Technology

also a representation of the expert's intention. At test time, the network can be given corresponding commands, which resolve the ambiguity in the perceptuomotor mapping and allow the trained model to be controlled by a passenger or a topological planner, just as mapping applications and passengers provide turn-by-turn directions to human drivers. The trained network is thus freed from the task of planning and can devote its representational capacity to driving. This enables scaling imitation learning to vision-based driving in complex urban environments.

We evaluate the presented approach in realistic simulations of urban driving and on a 1/5 scale robotic truck. Both systems are shown in Figure 1. Simulation allows us to thoroughly analyze the importance of different modeling decisions, carefully compare the approach to relevant baselines, and conduct detailed ablation studies. Experiments with the physical system demonstrate that the approach can be successfully deployed in the physical world. Recordings of both systems are provided in the supplementary video.

II. Related Work

Imitation learning has been applied to a variety of tasks, including articulated motion [2], [28], [11], autonomous flight [1], [13], [30], modeling navigational behavior [37], [38], off-road driving [22], [32], and road following [4], [6], [27], [36]. Technically, these applications differ in the input representation (raw sensory input or hand-crafted features), the control signal being predicted, the learning algorithms, and the learned representations. Most relevant to our work are the systems of Pomerleau [27], LeCun et al. [22], and Bojarski et al. [4], who used ground vehicles and trained deep networks to predict the driver's actions from camera input. These studies focused on purely reactive tasks, such as lane following or obstacle avoidance. In comparison, we develop a command-conditional formulation that enables the application of imitation learning to more complex urban driving. Another difference is that we learn to control not only steering but also acceleration and braking, enabling the model to assume full control of the car.

The decomposition of complex tasks into simpler subtasks has been studied from several perspectives. In robotics, movement primitives have been used as building blocks for advanced motor skills [17], [26]. Movement primitives represent a simple motion, such as a strike or a throw, by a parameterized dynamical system. In comparison, the policies we consider have much richer parameterizations and address more complex sensorimotor tasks that couple perception and control, such as finding the next opportunity to turn right and then making the turn while avoiding dynamic obstacles.

In reinforcement learning, hierarchical approaches aim to construct multiple levels of temporally extended subpolicies [3]. The options framework is a prominent example of such hierarchical decomposition [33]. Basic motor skills that are learned in this framework can be transferred across tasks [19]. Hierarchical approaches have also been combined with deep learning and applied to raw sensory input [20]. In these works, the main aim is to learn purely from experience and discover hierarchical structure automatically. This is hard and is in general an open problem, particularly for sensorimotor skills with the complexity we consider. In contrast, we focus on imitation learning, and we provide additional information on the expert's intentions during demonstration. This formulation makes the learning problem more tractable and yields a human-controllable policy.

Adjacent to hierarchical methods is the idea of learning multi-purpose and parameterized controllers. Parameterized goals have been used to train motion controllers in robotics [7], [8], [18]. Schaul et al. [31] proposed a general framework for reinforcement learning with parameterized value functions, shared across states and goals. Dosovitskiy and Koltun [9] studied families of parameterized goals in the context of navigation in three-dimensional environments. Javdani et al. [15] studied a scenario where a robot assists a human and changes its behavior depending on its estimate of the human's goal. Our work shares the idea of training a conditional controller, but differs in the model architecture, the application domain (vision-based autonomous driving), and the learning method (conditional imitation learning).

Autonomous driving is the subject of intensive research [25]. Broadly speaking, approaches differ in their level of modularity. On one side are highly tuned systems that deploy an array of computer vision algorithms to create a model of the environment, which is then used for planning and control [12]. On the opposite side are end-to-end approaches that train function approximators to map sensory input to control commands [4], [27], [36]. Our approach is on the end-to-end side of the spectrum, but in addition to sensory input the controller is provided with commands that specify the driver's intent. This resolves some of the ambiguity in the perceptuomotor mapping and creates a communication channel that can be used to guide the autonomous car as one would guide a chauffeur.

Human guidance of robot actions has been studied extensively [5], [14], [24], [34], [35]. These works tackle the challenging problem of parsing natural language instructions. Our work does not address natural language communication; we limit commands to a predefined vocabulary such as "turn right at the next intersection", "turn left at the next intersection", and "keep straight". On the other hand, our work deals with end-to-end vision-based driving using deep networks. Systems in this domain have been limited to imitating the expert without the ability to naturally accept commands after deployment [4], [6], [27], [36]. We introduce such ability into deep networks for end-to-end vision-based driving.

III. Conditional Imitation Learning

We begin by describing the standard imitation learning setup and then proceed to our command-conditional formulation. Consider a controller that interacts with the environment over discrete time steps. At each time step t, the controller receives an observation $\mathbf{o}_t$ and takes an action $\mathbf{a}_t$. The basic idea behind imitation learning is to train a controller that mimics an expert. The training data is a set of observation-action pairs $\mathcal{D} = \{\langle \mathbf{o}_i, \mathbf{a}_i \rangle\}_{i=1}^{N}$ generated by the expert. The assumption is that the expert is successful at performing the

task of interest and that a controller trained to mimic the expert will also perform the task well. This is a supervised learning problem, in which the parameters $\boldsymbol{\theta}$ of a function approximator $F(\mathbf{o}; \boldsymbol{\theta})$ must be optimized to fit the mapping of observations to actions:

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \sum_i \ell\big(F(\mathbf{o}_i; \boldsymbol{\theta}), \mathbf{a}_i\big). \tag{1}$$

An implicit assumption behind this formulation is that the expert's actions are fully explained by the observations. That is, there exists a function E that maps observations to the expert's actions: $\mathbf{a}_i = E(\mathbf{o}_i)$. If this assumption holds, a sufficiently expressive approximator will be able to fit the function E given enough data. This explains the success of imitation learning on tasks such as lane following. However, in more complex scenarios the assumption that the mapping of observations to actions is a function breaks down. Consider a driver approaching an intersection. The drivers's subsequent actions are not explained by the observations, but are additionally affected by the driver's internal state, such as the intended destination. The same observations could lead to different actions, depending on this latent state. This could be modeled as stochasticity, but a stochastic formulation misses the underlying causes of the behavior. Moreover, even if a controller trained to imitate demonstrations of urban driving did learn to make turns and avoid collisions, it would still not constitute a useful driving system. It would wander the streets, making arbitrary decisions at intersections. A passenger in such a vehicle would not be able to communicate the intended direction of travel to the controller, or give it commands regarding which turns to take.

To address this, we begin by explicitly modeling the expert's internal state by a vector $\mathbf{h}$, which together with the observation explains the expert's action: $\mathbf{a}_i = E(\mathbf{o}_i, \mathbf{h}_i)$. Vector $\mathbf{h}$ can include information about the expert's intentions, goals, and prior knowledge. The standard imitation learning objective can then be rewritten as

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \sum_i \ell\big(F(\mathbf{o}_i; \boldsymbol{\theta}), E(\mathbf{o}_i, \mathbf{h}_i)\big). \tag{2}$$

It is now clear that the expert's action is affected by information that is not provided to the controller F.

We expose the latent state $\mathbf{h}$ to the controller by introducing an additional command input: $\mathbf{c} = \mathbf{c}(\mathbf{h})$. At training time, the command $\mathbf{c}$ is provided by the expert. It need not constitute the entire latent state $\mathbf{h}$, but should provide useful information about the expert's decision-making. For example, human drivers already use turn signals to communicate their intent when approaching intersections; these turn signals can be used as commands in our formulation. At test time, commands can be used to affect the behavior of the controller. These test-time commands can come from a human user or a planning module. In urban driving, a typical command would be "turn right at the next intersection", which can be provided by a navigation system or a passenger. The training dataset becomes $\mathcal{D} = \{\langle \mathbf{o}_i, \mathbf{c}_i, \mathbf{a}_i \rangle\}_{i=1}^N$.

The command-conditional imitation learning objective is

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \sum_i \ell\big(F(\mathbf{o}_i, \mathbf{c}_i; \boldsymbol{\theta}), \mathbf{a}_i\big). \tag{3}$$

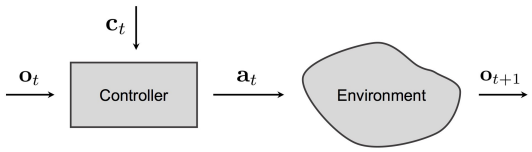


Fig. 2. High-level overview. The controller receives an observation $\mathbf{o}_t$ from the environment and a command $\mathbf{c}_t$. It produces an action $\mathbf{a}_t$ that affects the environment, advancing to the next time step.

In contrast with objective (2), the learner is informed about the expert's latent state and can use this additional information in predicting the action. This setting is illustrated in Figure 2.

IV. Methodology

We now describe a practical implementation of command-conditional imitation learning.

A. Network Architecture

Assume that each observation $\mathbf{o} = \langle \mathbf{i}, \mathbf{m} \rangle$ comprises an image $\mathbf{i}$ and a low-dimensional vector $\mathbf{m}$ that we refer to as measurements, following Dosovitskiy and Koltun [9]. The controller F is represented by a deep network. The network takes the image $\mathbf{i}$, the measurements $\mathbf{m}$, and the command $\mathbf{c}$ as inputs, and produces an action $\mathbf{a}$ as its output. The action space can be discrete, continuous, or a hybrid of these. In our driving experiments, the action space is continuous and two-dimensional: steering angle and acceleration. The acceleration can be negative, which corresponds to braking or driving backwards. The command $\mathbf{c}$ is a categorical variable represented by a one-hot vector.

We study two approaches to incorporating the command $\mathbf{c}$ into the network. The first architecture is illustrated in Figure 3(a). The network takes the command as an input, alongside the image and the measurements. These three inputs are processed independently by three modules: an image module $I(\mathbf{i})$, a measurement module $M(\mathbf{m})$, and a command module $C(\mathbf{c})$. The image module is implemented as a convolutional network, the other two modules as fully-connected networks. The outputs of these modules are concatenated into a joint representation:

$$\mathbf{j} = J(\mathbf{i}, \mathbf{m}, \mathbf{c}) = \langle I(\mathbf{i}), M(\mathbf{m}), C(\mathbf{c}) \rangle\,. \tag{4}$$

The control module, implemented as a fully-connected network, takes this joint representation and outputs an action $A(\mathbf{j})$. We refer to this architecture as `command input`. It is applicable to both continuous and discrete commands of arbitrary dimensionality. However, the network is not forced to take the commands into account, which can lead to suboptimal performance in practice.

We therefore designed an alternative architecture, shown in Figure 3(b). The image and measurement modules are as described above, but the command module is removed. Instead, we assume a discrete set of commands $\mathcal{C} = \{\mathbf{c}^0, \ldots, \mathbf{c}^K\}$ and introduce a specialist branch A^i for each of the commands $\mathbf{c}^i$. The command $\mathbf{c}$ acts as a switch that selects which branch is used at any given time. (One of the branches is responsible for acting when no command is provided.

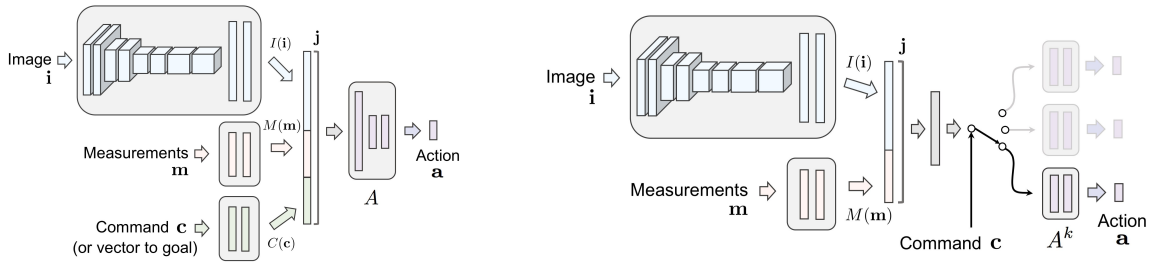


Fig. 3. Two network architectures for command-conditional imitation learning. (a) `command input`: the command is processed as input by the network, together with the image and the measurements. The same architecture can be used for goal-conditional learning (one of the baselines in our experiments), by replacing the command by a vector pointing to the goal. (b) `branched`: the command acts as a switch that selects between specialized sub-modules.

The lack of an explicit command is represented as a default command $\mathbf{c}^0$.) The output of the network is thus

$$F(\mathbf{i}, \mathbf{m}, \mathbf{c}^i) = A^i(J(\mathbf{i}, \mathbf{m})). \tag{5}$$

We refer to this architecture as `branched`. The branches A^i are forced to learn sub-policies that correspond to different commands. In a driving scenario, one module might specialize in lane following, another in right turns, and a third in left turns. All modules share the perception stream.

B. Network Details

For all controllers, the observation $\mathbf{o}$ is the currently observed image at 200×88 pixel resolution. For the measurement $\mathbf{m}$, we used the current speed of the car, if available (in the physical system the speed estimates were very noisy and we refrained from using them). All networks are composed of modules with identical architectures (e.g., the ConvNet architecture is the same in all conditions). The differences are in the configuration of modules and branches as can be seen in Figure 3. The image module consists of 8 convolutional and 2 fully connected layers. The convolution kernel size is 5 in the first layer and 3 in the following layers. The first, third, and fifth convolutional layers have a stride of 2. The number of channels increases from 32 in the first convolutional layer to 256 in the last. Fully-connected layers contain 512 units each. All modules with the exception of the image module are implemented as standard multilayer perceptrons. We used ReLU nonlinearities after all hidden layers, performed batch normalization after convolutional layers, applied 50% dropout after fully-connected hidden layers, and used 20% dropout after convolutional layers.

Actions are two-dimensional vectors that collate steering angle and acceleration: $\mathbf{a} = \langle s, a \rangle$. Given a predicted action $\mathbf{a}$ and a ground truth action $\mathbf{a}_{\text{gt}}$, the per-sample loss function is defined as

$$\begin{aligned} \ell(\mathbf{a}, \mathbf{a}_{\text{gt}}) &= \ell\big(\langle s, a \rangle, \langle s_{\text{gt}}, a_{\text{gt}} \rangle\big) \\ &= \|s - s_{\text{gt}}\|^2 + \lambda_a \|a - a_{\text{gt}}\|^2 . \end{aligned} \tag{6}$$

All models were trained using the Adam solver [16] with minibatches of 120 samples and an initial learning rate of 0.0002. For the command-conditional models, minibatches were constructed to contain an equal number of samples with each command.

C. Training Data Distribution

When performing imitation learning, a key decision is how to collect the training data. The simplest solution is to collect trajectories from natural demonstrations of an expert performing the task. This typically leads to unstable policies, since a model that is only trained on expert trajectories may not learn to recover from disturbance or drift [23], [29].

To overcome this problem, training data should include observations of recoveries from perturbations. In DAgger [29], the expert remains in the loop during the training of the controller: the controller is iteratively tested and samples from the obtained trajectories are re-labeled by the expert. In the system of Bojarski et al. [4], the vehicle is instrumented to record from three cameras simultaneously: one facing forward and the other two rotated to the left and to the right. Recordings from the rotated cameras are added to the training set – with appropriately adjusted control signals – to simulate recovery from drift.

In this paper we adopt a three-camera setup similar to Bojarski et al. [4]. However, we have found that the policies learned with this setup are not sufficiently robust. Therefore, to further augment the training dataset, we record some of the data while injecting noise into the expert's control signal and letting the expert recover from these perturbations. This is akin to the recent approach of Laskey et al. [21], but instead of i.i.d. noise we inject temporally correlated noise designed to simulate gradual drift away from the desired trajectory. An example is shown in Figure 4. For training, we use the driver's corrective response to the injected noise (not the noise itself). This provides the controller with demonstrations of recovery from drift and unexpected disturbances, but does not contaminate the training set with demonstrations of veering away from desired behavior.

D. Data Augmentation

We found data augmentation to be crucial for good generalization. We perform augmentation online during network training. For each image to be presented to the network, we apply a random subset of a set of transformations with randomly sampled magnitudes. Transformations include change in contrast, brightness, and tone, as well as addition of Gaussian blur, Gaussian noise, salt-and-pepper noise, and region dropout (masking out a random set of rectangles in the image, each rectangle taking roughly 1% of image area).

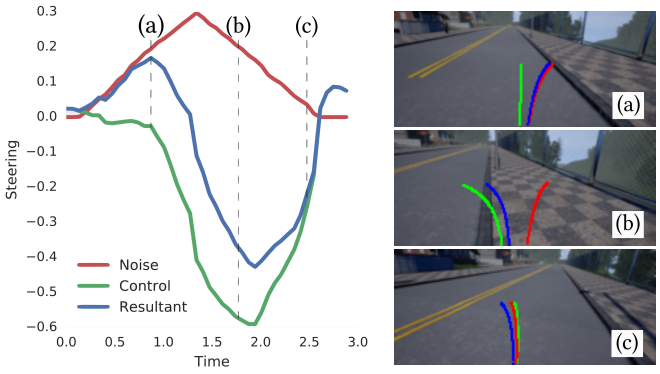


Fig. 4. Noise injection during data collection. We show a fragment from an actual driving sequence from the training set. The plot on the left shows steering control versus time. In the plot, the red curve is an injected triangular noise signal, the green curve is the driver's steering signal, and the blue curve is the steering signal provided to the car, which is the sum of the driver's control and the noise. Images on the right show the driver's view at three points in time (trajectories overlaid post-hoc for visualization). Between times 0 and roughly 1.0, the noise produces a drift to the right, as illustrated in image (a). This triggers a human reaction, from 1.0 to 2.5 seconds, illustrated in (b). Finally, the car recovers from the disturbance, as shown in (c). Only the driver-provided signal (green curve on the left) is used for training.

No geometric augmentations such as translation or rotation were applied, since control commands are not invariant to these transformations.

V. System Setup

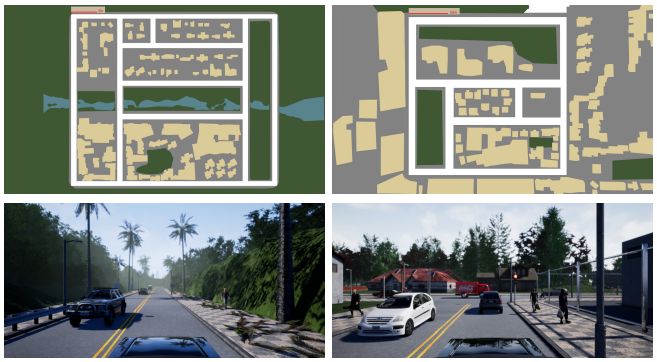
We evaluated the presented approach in a simulated urban environment and on a physical system – a 1/5 scale truck. In both cases, the observations (images) are recorded by one central camera and two lateral cameras rotated by 30 degrees with respect to the center. The recorded control signal is two-dimensional: steering angle and acceleration. The steering angle is scaled between -1 and 1, with extreme values corresponding to full left and full right, respectively. The acceleration is also scaled between -1 and 1, where 1 corresponds to full forward acceleration and -1 to full reverse acceleration.

In addition to the observations (images) and actions (control signals), we record commands provided by the driver. We use a set of four commands: `continue` (follow the road), `left` (turn left at the next intersection), `straight` (go straight at the next intersection), and `right` (turn right at the next intersection). In practice, we represent these as one-hot vectors.

During training data collection, when approaching an intersection the driver uses buttons on a physical steering wheel (when driving in simulation) or on the remote control (when operating the physical truck) to indicate the command corresponding to the intended course of action. The driver indicates the command when the intended action becomes clear, akin to turn indicators in cars or navigation instructions provided by mapping applications. This way we collect realistic data that reflects how a higher level planner or a human could direct the system.

A. Simulated Environment

We use CARLA [10], an urban driving simulator, to corroborate design decisions and evaluate the proposed approach in a dynamic urban environment with traffic. CARLA is an open-source simulator implemented using Unreal Engine 4. It contains two professionally designed towns with buildings, vegetation, and traffic signs, as well as vehicular and pedestrian traffic. Figure 5 provides maps and sample views of Town 1, used for training, and Town 2, used exclusively for testing.



Town 1 (training) Town 2 (testing)

Fig. 5. Simulated urban environments. Town 1 is used for training (left), Town 2 is used exclusively for testing (right). Map on top, view from onboard camera below. Note the difference in visual style.

In order to collect training data, a human driver is presented with a first-person view of the environment (center camera) at a resolution of 800×600 pixels. The driver controls the simulated vehicle using a physical steering wheel and pedals, and provides command input using buttons on the steering wheel. The driver keeps the car at a speed below 60 kilometers per hour and strives to avoid collisions with cars and pedestrians, but ignores traffic lights and stop signs. We record images from the three simulated cameras, along with other measurements such as speed and the position of the car. The images are cropped to remove part of the sky. CARLA also provides extra information such as distance travelled, collisions, and the occurrence of infractions such as drift onto the opposite lane or the sidewalk. This information is used in evaluating different controllers.

B. Physical System

The setup of the physical system is shown in Figure 6. We equipped an off-the-shelf 1/5 scale truck (Traxxas Maxx) with an embedded computer (Nvidia TX2), three low-cost webcams, a flight controller (Holybro Pixhawk) running the APMRover firmware, and supporting electronics. The TX2 acquires images from the webcams and shares a bidirectional communication channel with the Pixhawk. The Pixhawk receives controls from either the TX2 or a human driver and converts them to low-level PWM signals for the speed controller and steering servo of the truck.

1) Data collection: During data collection the truck is driven by a human. The images from all three cameras are synchronized with the control signals and with GPS and IMU data from the Pixhawk, and recorded to disk. The control

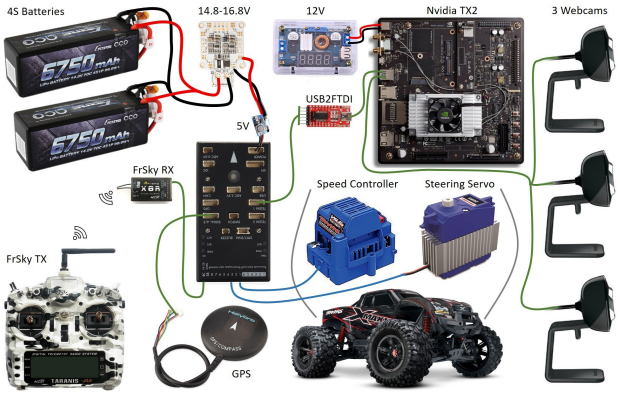


Fig. 6. Physical system setup. Red/black indicate +/- power wires, green indicates serial data connections, and blue indicates PWM control signals.

signals are passed through the TX2 to support noise injection as described in Section IV-C. In addition, routing the control through the TX2 ensures a similar delay in the training data as during test time. For the physical system we use only three command inputs (`left`, `straight`, `right`), since only a three-way switch is available on the remote control.

2) Model evaluation: At test time the trained model is evaluated on the TX2 in real time. It receives images from the central webcam and commands (`left`, `straight`, `right`) from the remote control. Figure 7(b) shows an example image from the central camera. The network predicts the appropriate controls in an end-to-end fashion based on only the current image and the provided command. The predicted control is forwarded to the Pixhawk, which controls the car accordingly by sending the appropriate PWM signals to the speed controller and steering servo.



(a) Left camera (b) Central camera (c) Right camera

Fig. 7. Images from the three cameras on the truck. All three cameras are used for training, with appropriately adjusted steering commands. Only the central camera is used at test time.

VI. Experiments

A. Simulated Environment

1) Experimental setup: The use of the CARLA simulator enables running the evaluation in an episodic setup. In each episode, the agent is initialized at a new location and has to drive to a given destination point, given high-level turn commands from a topological planner. An episode is considered successful if the agent reaches the goal within a fixed time interval. In addition to success rate, we measured driving quality by recording the average distance travelled without infractions (collisions or veering outside the lane).

The two CARLA towns used in our experiments are illustrated in Figure 5 and in the supplementary video. Town 1 is used for training, Town 2 is used exclusively for testing. For evaluation, we used 50 pairs of start and goal locations set at least 1 km apart, in each town.

Model	Success rate		Km per infraction	
	Town 1	Town 2	Town 1	Town 2
Non-conditional	20%	26%	5.76	0.89
Goal-conditional	24%	30%	1.87	1.22
Ours branched	**88%**	**64%**	2.34	1.18
Ours cmd. input	78%	52%	3.97	1.30
Ours no noise	56%	22%	1.31	0.54
Ours no aug.	80%	0%	4.03	0.36
Ours shallow net	46%	14%	0.96	0.42

Table 1. Results in the simulated urban environment. We compare the presented method to baseline approaches and perform an ablation study. We measure the percentage of successful episodes and the average distance (in km) driven between infractions. Higher is better in both cases. The proposed `branched` architecture outperforms the baselines and the ablated versions.

Our training dataset comprises 2 hours of human driving in Town 1. Only 10%, or roughly 12 minutes, contain demonstrations with injected noise. We have found that collecting training data with strong injected noise was quite tiring for the trainer. On the other hand, a relatively small amount of such data proved very effective in stabilizing the learned policy.

2) Results: We compare the `branched` command-conditional architecture, as shown in Figure 3(b), with two baseline approaches, as well as several ablated versions of the full architecture. The two baselines are standard imitation learning and goal-conditioned imitation learning. In standard (non-conditional) imitation learning, the action $\mathbf{a}$ is predicted from the observation $\mathbf{o}$ and the measurement $\mathbf{m}$. In the goal-conditional variant, the controller is additionally provided with a vector pointing to the goal, in the car's coordinate system (the architecture follows Figure 3(a)). Ablated versions include: a network with the `command input` architecture instead of `branched` (see Figure 3), and three variants of the `branched` network: trained without noise-injected data, trained without data augmentation, and implemented with a shallower network.

The results are summarized in Table 1. The controller that is trained using standard imitation learning only completes 20% of the episodes in Town 1 and 24% in Town 2, which is not surprising given its ignorance of the goal. More interestingly, the goal-conditional controller, which is provided with an accurate vector to the goal at every time step during both training and at test time, is performing only slightly better than the non-conditional controller, successfully completing 24% of the episodes in Town 1 and 30% in Town 2. Qualitatively, this controller eventually veers off the road attempting to shortcut to the goal. This also decreases the number of kilometers the controller is able to traverse without infractions. A simple feed-forward network does not automatically learn to convert a vector pointing to the goal into a sequence of turns.

The proposed `branched` command-conditional controller performs significantly better than the baseline methods

in both towns, successfully completing 88% of the episodes in Town 1 and 64% in Town 2. In terms of distance travelled without infractions, in Town 2 the method is on par with baselines, while in Town 1 it is outperformed by the non-conditional model. This difference is misleading: the non-conditional model drives more cleanly because it is not constrained to travel towards the goal and therefore typically takes a simpler route at each intersection.

The ablation study shown in the bottom part of Table 1 reveals that all components of the proposed system are important for good performance. The `branched` architecture reaches the goal more reliably than the `command input` one. The addition of even a small amount of training data with noise in the steering dramatically improves the performance. (Recall that we have only 12 minutes of noisy data out of the total of 2 hours.) Careful data augmentation is crucial for generalization, even within Town 1, but much more so in the previously unseen Town 2: the model without data augmentation was not able to complete a single episode there. Finally, a sufficiently deep network is needed to learn the perceptuomotor mapping in the visually rich and complex simulated urban environment.

B. Physical System

1) Experimental setup: The training dataset consists of 2 hours of driving the truck via remote control in a residential area. Figure 8 shows a map with the route on which the vehicle was evaluated. The route includes a total of 15 turns with an equal number of `left`, `straight`, and `right`.

We measure the performance in terms of missed turns, interventions, and time to complete the course. If the robotic vehicle misses a turn for the first time, it is rerouted to get a second chance to do the turn. If it manages to do the turn the second time, this is not counted as a missed turn but increases the time taken to complete the route. However, if the vehicle misses the turn for the second time, this is counted as a missed turn and we intervene to drive the vehicle through the turn manually. Besides missed turns, we also intervene if the vehicle goes off the road for more than five seconds or if it collides with an obstacle. The models were all evaluated in overcast weather conditions. The majority of training data was collected in sunny weather.

2) Main results: We select the most important comparisons from the extensive evaluation performed in simulation (Section VI-A) and perform them on the physical system. Table 2 shows the results of several variants of command-conditional imitation learning: `branched` and `command input` architectures, as well as two ablated models, trained without data augmentation or without noise-injected data. It is evident that the `branched` architecture achieves the best performance. The ablation experiments show the impact of our noise injection method and augmentation strategy. The model trained without noise injection is very unstable, as indicated by the average number of interventions rising from 0.67 to 8.67. Moreover, it misses almost 25% of the turns and takes double the time to complete the course. The model trained without data augmentation fails completely. The truck misses most turns and very frequently leaves the lane resulting in almost 40 interventions. It takes more than four times longer to complete the course. This extreme degradation highlights the importance of generalization in real world settings with constantly changing environmental conditions such as weather and lighting. Proper data augmentation dramatically improves performance given limited training data.

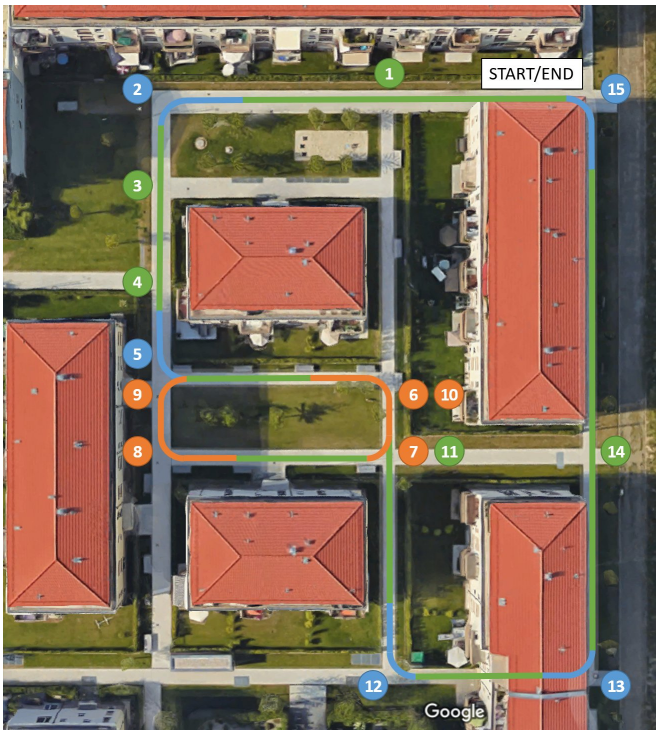


Fig. 8. A map of the primary route used for testing the physical system. Intersections traversed by the truck are numbered according to their order along the route. Colors indicate commands provided to the vehicle when it approaches the intersection: blue = `left`, green = `straight`, orange = `right`.

Model	Missed turns	Interventions	Time
Ours branched	**0%**	**0.67**	**2:19**
Ours cmd. input	11.1%	2.33	4:13
Ours no noise	24.4%	8.67	4:39
Ours no aug.	73%	39	10:41

Table 2. Results on the physical system. Lower is better. We compare the `branched` model to the simpler `command input` architecture and to ablated versions (without noise injection and without data augmentation). Average performance across 3 runs is reported for all models except for "Ours no aug.", for which we only performed 1 run to avoid breaking the truck.

3) Generalization to new environments: Beyond the implicit generalization to varying weather conditions (Section VI-B.2), we also evaluate qualitatively how well the model generalizes to previously unseen environments with very different appearance. To this end, we run the truck in three environments shown in Figure 9. The truck is able to consistently follow the lane in all tested environments and is responsive to commands. These and other experiments are shown in the supplementary video.



Fig. 9. Testing in new environments with very different appearance.

VII. Discussion

We proposed command-conditional imitation learning: an approach to learning from expert demonstrations of low-level controls and high-level commands. At training time, the commands resolve ambiguities in the perceptuomotor mapping, thus facilitating learning. At test time, the commands serve as a communication channel that can be used to direct the controller.

We applied the presented approach to vision-based driving of a physical robotic vehicle and in realistic simulations of dynamic urban environments. Our results show that the command-conditional formulation significantly improves performance in both scenarios.

While the presented results are encouraging, they also reveal that significant room for progress remains. In particular, more sophisticated and higher-capacity architectures along with larger datasets will be necessary to support autonomous urban driving on a large scale. We hope that the presented approach to making driving policies more controllable will prove useful in such deployment.

Our work has not addressed human guidance of autonomous vehicles using natural language: a mode of human-robot communication that has been explored in the literature [5], [14], [24], [34], [35]. We leave unstructured natural language communication with autonomous vehicles as an important direction for future work.

References

[1] P. Abbeel, A. Coates, and A. Y. Ng. Autonomous helicopter aerobatics through apprenticeship learning. *International Journal of Robotics Research*, 29(13), 2010.

[2] B. Argall, S. Chernova, M. M. Veloso, and B. Browning. A survey of robot learning from demonstration. *Robotics and Autonomous Systems*, 57(5), 2009.

[3] A. G. Barto and S. Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, 13(1-2), 2003.

[4] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba. End to end learning for self-driving cars. *arXiv:1604.07316*, 2016.

[5] A. Broad, J. Arkin, N. Ratliff, T. Howard, and B. Argall. Real-time natural language corrections for assistive robotic manipulators. *International Journal of Robotics Research*, 2017.

[6] C. Chen, A. Seff, A. L. Kornhauser, and J. Xiao. DeepDriving: Learning affordance for direct perception in autonomous driving. In *ICCV*, 2015.

[7] B. C. da Silva, G. Konidaris, and A. G. Barto. Learning parameterized skills. In *ICML*, 2012.

[8] M. P. Deisenroth, P. Englert, J. Peters, and D. Fox. Multi-task policy search for robotics. In *ICRA*, 2014.

[9] A. Dosovitskiy and V. Koltun. Learning to act by predicting the future. In *ICLR*, 2017.

[10] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun. CARLA: An open urban driving simulator. In *Conference on Robot Learning (CoRL)*, 2017.

[11] P. Englert, A. Paraschos, J. Peters, and M. P. Deisenroth. Model-based imitation learning by probabilistic trajectory matching. In *ICRA*, 2013.

[12] U. Franke. Autonomous driving. In *Computer Vision in Vehicle Technology*. 2017.

[13] A. Giusti, J. Guzzi, D. Ciresan, F.-L. He, J. P. Rodriguez, F. Fontana, M. Faessler, C. Forster, J. Schmidhuber, G. Di Caro, D. Scaramuzza, and L. Gambardella. A machine learning approach to visual perception of forest trails for mobile robots. *IEEE Robotics and Automation Letters*, 2016.

[14] S. Hemachandra, F. Duvallet, T. M. Howard, N. Roy, A. Stentz, and M. R. Walter. Learning models for following natural language directions in unknown environments. In *ICRA*, 2015.

[15] S. Javdani, S. S. Srinivasa, and J. A. Bagnell. Shared autonomy via hindsight optimization. In *RSS*, 2015.

[16] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[17] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *International Journal of Robotics Research*, 32(11), 2013.

[18] J. Kober, A. Wilhelm, E. Oztop, and J. Peters. Reinforcement learning to adjust parametrized motor primitives to new situations. *Autonomous Robots*, 33(4), 2012.

[19] G. Konidaris, S. Kuindersma, R. A. Grupen, and A. G. Barto. Robot learning from demonstration by constructing skill trees. *International Journal of Robotics Research*, 31(3), 2012.

[20] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. B. Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *NIPS*, 2016.

[21] M. Laskey, A. Dragan, J. Lee, K. Goldberg, and R. Fox. Dart: Optimizing noise injection in imitation learning. In *Conference on Robot Learning (CoRL)*, 2017.

[22] Y. LeCun, U. Muller, J. Ben, E. Cosatto, and B. Flepp. Off-road obstacle avoidance through end-to-end learning. In *NIPS*, 2005.

[23] S. Levine and V. Koltun. Guided policy search. In *ICML*, 2013.

[24] C. Matuszek, L. Bo, L. Zettlemoyer, and D. Fox. Learning from unscripted deictic gesture and language for human-robot interactions. In *AAAI*, 2014.

[25] B. Paden, M. Cáp, S. Z. Yong, D. S. Yershov, and E. Frazzoli. A survey of motion planning and control techniques for self-driving urban vehicles. *IEEE Transactions on Intelligent Vehicles*, 1(1), 2016.

[26] P. Pastor, H. Hoffmann, T. Asfour, and S. Schaal. Learning and generalization of motor skills by learning from demonstration. In *ICRA*, 2009.

[27] D. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *NIPS*, 1988.

[28] N. D. Ratliff, J. A. Bagnell, and S. S. Srinivasa. Imitation learning for locomotion and manipulation. In *International Conference on Humanoid Robots*, 2007.

[29] S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.

[30] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert. Learning monocular reactive UAV control in cluttered natural environments. In *ICRA*, 2013.

[31] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *ICML*, 2015.

[32] D. Silver, J. A. Bagnell, and A. Stentz. Learning from demonstration for autonomous navigation in complex unstructured terrain. *International Journal of Robotics Research*, 29(12), 2010.

[33] R. S. Sutton, D. Precup, and S. P. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2), 1999.

[34] S. Tellex, T. Kollar, S. Dickerson, M. R. Walter, A. G. Banerjee, S. J. Teller, and N. Roy. Understanding natural language commands for robotic navigation and mobile manipulation. In *AAAI*, 2011.

[35] M. R. Walter, S. Hemachandra, B. Homberg, S. Tellex, and S. J. Teller. Learning semantic maps from natural language descriptions. In *RSS*, 2013.

[36] J. Zhang and K. Cho. Query-efficient imitation learning for end-to-end simulated driving. In *AAAI*, 2017.

[37] B. D. Ziebart, A. L. Maas, J. A. Bagnell, and A. K. Dey. Maximum entropy inverse reinforcement learning. In *AAAI*, 2008.

[38] B. D. Ziebart, A. L. Maas, A. K. Dey, and J. A. Bagnell. Navigate like a cabbie: Probabilistic reasoning from observed context-aware behavior. In *UbiComp*, 2008.